



THE CATALA SYNTAX

english

Literate programming

# Title	Heading
### Sub-subtitle	
``catala      ``catala-metadata ```	Code / metadata block
> Module Mdl	Module declaration
> Using Mdl as M	Module import+alias
> Include: foo.catala_en	File inclusion
``catala-test-cli \$ catala interpret --scope Scope1 ```	Command-line interface test

Literals and types

true	false	boolean
65,536		integer
65536.262144	37%	decimal
\$1,234,567.89		money
2026-07-30		date
254 day -4 month 1 year		duration
Case1 content 12 Case2		Enum1
Struct1 { -- fld1: 9 -- fld2: 7% }		Struct1
Present content \$34 Absent		optional of money
[12; 24; 36]		list of integer
[]		list of
(2012-02-03 , \$30, 1%)		anything of type t
(date,money,decimal)		
f of x, y equals y * x / \$12.0		decimal depends on x content money, y content decimal

Operators and built-ins

not a	a and b	
a or b	# "or otherwise"	Logical operators
a xor b	# exclusive or	
- a	a + b	a - b
a * b	a / b	
a = b	a < b	a <= b
a != b	a > b	a >= b
decimal of 44		
money of 23.15		Conversions
round of \$9.99		Rounding
Date.get_month of ...		
Date.first_day_of_month of ...		Date parts

Metadata declaration

declaration structure Struct1: data fld1 content integer data fld2 content decimal declaration enumeration Enum1: -- Case1 content integer -- Case2	Structure declaration
## Documentation for Scope1 #[test]	Enumeration declaration
documentation string	Documentation string
test scope annotation	Test scope annotation
declaration scope Scope1: internal var1 content integer state before state after	Scope declaration
internal var2 condition	Scope variable
sub1 scope Scope0	State transitions
output var3 content ...	Condition
input var4 content ...	Sub-scope declaration
input output var5 content ...	
context var6 content ...	
context output var7 content ...	
output sub2 scope Scope0	
declaration const content decimal equals 17.1	Input-output qualifiers
declaration square content decimal depends on x content decimal equals x * x	
	Global definition
	Global function definition

Expressions

let x equals 36 - 5 in ...	Local definition
match expr with pattern	
-- Case1 content x : ...	
-- Case2 : ...	
-- anything : ...	
#[error.message = "err"] impossible	Pattern matching
if ... then ... else ...	Unreachable code
#[debug.print = "message"] expr	Conditional
expr with pattern Case1 content x and x >= 2	Debug annotation
struc1 but replace { -- fld2: 8% }	Pattern test
struc1.fld2 tuple1.2	(optional: var. binding)
sub1.var0	Field replacement
f of \$44.50, 1/3	Field, tuple element, subscope variable
output of Scope1 with { -- fld1: 9 -- fld2: 15% }	Function call
var1 state before	Direct scope call
assertion x > 0 in ...	Variable state access
	Local assertion

Scope definition

scope Scope1: ...	Defining scope contents
scope Scope1 under condition var1 >= 2: ...	Use-wide condition
definition var1 equals ...	Unconditional def.
definition var1 under condition ...	Conditional definition
consequence equals ...	
rule var2 under condition var1 >= 2 consequence not fulfilled	Rule (definition for conditions)
definition f of x, y equals ...	Function def. or rule
label lbl1 definition var1 ...	Labeled def. or rule
exception lbl1 definition var1 ...	Exception to label
exception definition var1 ...	Exception to implicit
definition var1 state before equals ...	State definition
assertion var1 > 0	Scope-level assertion
date round down up	Date rounding mode

List operations

lst contains 3	Presence test
exists x among lst such that x > 2	Existence test
for all x among lst we have x > 2	For all test
map each x among lst to x + 2	Mapping
list of x among lst such that x > 2	Filter
map each x among lst such that x > 2 to x - 2	Filter + map
map each (x, y) among (lst1, lst2) to x + y	Multiple mapping
lst1 ++ lst2	Merge
number of lst	Count
maximum of lst or if list empty then -1	Extremum search (optional default)
content of x among lst such that x.fld1 = ...	First element satisfying the condition
content of x among lst such that x * x is minimum or if list empty then -1	Arg-extremum search (optional default)
sort lst in in-decreasing order	Sorting
sort all x among lst in in-decreasing order of x.fld2 and then -x.fld1 ...	Sorting on criteria
combine all x among lst in acc initially 0 with acc + x	(optional sub-criteria)
	Fold (loop and accumulate)



Programmation littéraire

# Titre	En-têtes
### Sous-sous-titre	
# Article 1 JORFARTI000012345678	Référence au journal officiel
# Article 2 LEGIARTI000012345678	
# Décision 3 CETATEXT000012345678	
```catala       ```catala-metadata	Bloc de code / métadonnées
> Module Mdl	Déclaration de module
> Usage de Mdl en tant que M	Import de module
> Inclusion: foo.catala_en	Inclusion textuelle
```catala-test-cli	
\$ catala interpret --scope Chp1	Test intégré
```	

## Littéraux et types

vrai	faux	booléen
65536		entier
65536,262144	37%	décimal
1 234 567,89€		argent
2026-07-30		date
254 jour    -4 mois    1 an		durée
Cas1 contenu 12        Cas2		Enum1
Struct1 { -- chp1: 9 -- chp2: 7% }		Struct1
Présent contenu 34€    Absent		optionnel de argent
[ 12; 24; 36 ]		liste de entier
		liste de
		n'importe quel de type t
[ ]		
( 2012-02-03 , 30€, 1%)		(date,argent,décimal)
f de x, y égal à		décimal dépend de
y * x / 12,0€		x contenu argent, y contenu décimal

## Opérations

non a	a et b	
a ou b	# "ou à défaut"	Opérateurs logiques
a ou bien b	# ou exclusif	
- a	a + b	a - b
a * b	a / b	
a = b	a < b	a <= b
a != b	a > b	a >= b
décimal de 44		
argent de 23,15		
arrondi de 9,99€		
Date.accès_année de ...		
Date.premier_jour_du_mois de ...		

## Déclaration des métadonnées

déclaration structure Struct1:	Déclaration de structure
donnée chp1 contenu entier	
donnée chp2 contenu décimal	
déclaration énumération Enum1:	Déclaration d'énumération
-- Cas1 contenu entier	
-- Cas2	
## Documentation de Chp1	Texte de documentation
#[test]	Annot. champ de test
déclaration champ d'application Chp1:	Déclaration de champ
interne var1 contenu entier	Variable de champ
état avant	
état après	
interne var2 condition	Transitions d'état
sub1 champ d'application Chp0	Condition
résultat var3 contenu ...	s/s champ
entrée var4 contenu ...	
entrée résultat var5 contenu ...	
contexte var6 contenu ...	
contexte résultat var7 contenu ...	
résultat sub2	
champ d'application Chp0	
déclaration const contenu décimal	Qualificateurs d'entrée-sortie
égal à 17,1	
déclaration carré contenu décimal	
dépend de x contenu décimal	
égal à x * x	
	Définition globale
	Définition de fonction globale

## Expressions

soit x égal à 36 - 5 dans ...	Définition locale
selon expr sous forme	
-- Cas1 contenu x : ...	
-- n'importe quel : ...	
#[error.message = "err"] impossible	Calcul inaccessible
si ... alors ... sinon ...	Branchement
#[debug.print = "message"] expr	Annot. de debug
expr sous forme Cas1	Test de filtrage
contenu x et x >= 2	(optionnel: test contenu)
struc1 mais en remplaçant	Remplacement de champs
{ -- chp2: 8% }	
struc1.chp2            tuple1.2	Champ, élément de n-uplet, s/s-variable
f de 44,50€, 1/3	Appel de fonction
résultat de Chp1	Appel direct de champ d'application
avec { -- chp1: 9 -- chp2: 15% }	
var1 état avant	Accès à un état
assertion x > 0 dans ...	Assertion locale

## Définition de champ d'application

champ d'application Chp1: ...	Définition des membres
champ d'application Chp1	
sous condition var1 >= 2: ...	Avec condition générale
définition var1 égal à ...	Déf. inconditionnelle
définition var1	
sous condition ...	Définition conditionnelle
conséquence égal à ...	
règle var2	Règle
sous condition var1 >= 2	(définition de condition)
conséquence ·non· rempli	
définition f de x, y égal à ...	Déf./règle fonction
étiquette étq1 définition var1 ...	Déf./règle étiquetée
exception étq1 définition var1 ...	Exc. à déf. étiquetée
exception définition var1 ...	Exception à implicite
définition var1 état avant égal à ...	Définition d'états
assertion var1 > 0	Assertion de champ
date arrondi inf·supérieur	Mode arrondi dates

## Opérations sur les listes

lst contient 3	Test de présence
existe x parmi lst tel que x > 2	Test d'existence
pour tout x parmi lst on a x > 2	Test pour tout
transforme chaque x parmi lst	
en x - 2	Application un-à-un
liste de x parmi lst tel que x > 2	Filtrage
transforme chaque x parmi lst	
tel que x > 2 en x - 2	Filtrage + application
transforme chaque (x, y)	
parmi (lst1, lst2) en x + y	Application multiple
lst1 ++ lst2	Réunion
nombre de lst	Comptage
maximum de lst	Recherche d'extremum
ou si liste vide alors -1	(optionnel: défaut)
contenu de x parmi lst	Premier élément qui satisfait la condition
tel que x.chp1 = 0	
contenu de x parmi lst	
tel que x * x est minimum	Extremum selon critère
ou si liste vide alors -1	(optionnel: défaut)
trie lst par ordre décroissant	Tri
trie tout x parmi lst	Tri selon critère
par ordre décroissant de x.chp2	
puis -x.chp1 ...	(optionnel: sous-critère)
combine tout x parmi lst	
dans acc initialement 0	Fold (boucle avec accumulation)
avec acc + x	